

MECHENG 306 Design of Sensing and Actuating Systems

Project 1:

Design and Implementation of a Closed-Loop X-Y Plotter

Jonathan Stringer

July 14th, 2026

Design and Implementation of a Closed-Loop X-Y Plotter

1 Administration

1.1 Personnel

Project Organiser/Lecturer

Jonathan Stringer (Room 405.873, j.stringer@auckland.ac.nz).

Office Hours

Monday 1 – 2 pm (during duration of the project)

1.2 Project Objectives

- To design, implement, and analyse a complete, closed-loop mechatronic system.
- To develop and tune a robust motion control system for a 2-axis X-Y plotter.
- To create a structured and fault-tolerant software architecture for real-time control.
- To formally analyse the safety of a hypothetical hazardous system modification.

1.3 Learning Objectives

By the conclusion of this project, it is expected that you will have:

- Integrated mechanical hardware, sensors, actuators, and a microcontroller into a functional system.
- Implemented and tuned a PID (or similar) controller for precise position control.
- Developed a fault-tolerant software architecture using a Finite State Machine (FSM).
- Applied signal conditioning techniques to handle real-world sensor data.
- Developed a G-code parser to interface the system with a host computer.
- Applied formal safety analysis methods (HARA, FMEA) to evaluate the risks of a hypothetical hazardous system modification.

1.4 Project Weighting and Grouping

This project is worth 35% of the total mark for MECHENG 306. You will be working in groups of 4 from the same stream, where possible. You need to finalise your grouping during, or even better, before your designated first tutorial session or the end of week 1 (w/c 20th July). Then, you will need to sign up your group on Canvas. Those who have not signed up by the end of week 1 will be allocated a group. To sign up your group, log on to Canvas, go to “People” from the course navigation menu on the left, and on the “People” page select “Project Groups” and join a group.

1.5 Academic Integrity

All work submitted for this project must be the result of your group's own effort, with the exception of materials explicitly provided or permitted by the course staff. The sharing of code or detailed solutions between groups is a breach of academic integrity. All final code and analysis must be understood and justifiable by all team members.

2 Project Information

2.1 Design Specification

The goal of this design project is to implement a robust and reliable closed-loop control system for a 2-axis X-Y plotter. The system must be capable of receiving standard commands, executing precise motion, and operating in a safe, predictable manner.

The project will look at the entire mechatronic system design process, with a focus on control logic, software architecture, and system analysis. The following sections describe in more detail the specifications and constraints placed upon the design.

2.1.1 Provided Hardware

Each team will be provided with a standard kit of parts, which forms the physical basis of the design. No substitutions are permitted without express permission. The kit includes:

- **Chassis:** 1x Assembled X-Y plotter chassis.
- **Actuators:** 2x DC servo motors with integrated quadrature encoders (mounted on chassis).
- **Sensors:** 4x Mechanical limit switches (mounted on chassis).
- **Controller:** 1x Arduino Mega 2560.
- **Motor Driver:** 1x DFRobot L298P Motor Shield.
- **Connectivity:** USB Cable, breadboard, and jumper wires.
- **Power:** A 3-12 V 1,25 A switch mode power supply with barrel jack.

In addition, there will be duplicates of some components to aid in development. Data Sheets are available for all components where relevant on Canvas. The laboratory for timetabled sessions (either 405-559 or 405-569) is equipped with lab power supplies, oscilloscopes and multimeters.

2.1.2 Control Interface

The plotter must be controlled by a host computer via a standard serial connection. The system must parse and execute a subset of G-code commands sent through the Arduino IDE Serial Monitor or an equivalent terminal.

2.1.3 Functional Requirements

- **Homing:** The system must implement a G28 homing routine. Upon command, it must move each axis towards its origin until the respective limit switch is triggered, accurately defining this position as machine zero.
- **Linear Motion:** The system must implement a G1 command for linear moves. It must be able to move to specified X-Y coordinates in a coordinated fashion.
- **State Management:** The system must operate based on a clearly defined Finite State Machine (FSM), managing states such as IDLE, HOMING, MOVING, and FAULT.
- **Fault Tolerance:** The system must demonstrate basic fault tolerance. For example, if a limit switch is triggered unexpectedly during a move, the system must halt motion and enter a FAULT state.

2.1.4 Software and Signal Conditioning

All control logic will be implemented in C/C++ on the provided Arduino Mega.

- The software architecture must be non-blocking. The use of delay() is strictly prohibited for any task longer than a few microseconds.
- Robust functions must be developed to read quadrature encoder signals without missing steps.
- Software debouncing must be implemented for all mechanical limit switch inputs to prevent false readings.

2.1.5 Use of Pre-existing Software Libraries

A central aim of this project is for students to develop a fundamental understanding of control algorithms, signal processing, and real-time software architecture. Consequently, the implementation of core functionalities related to these learning objectives must be the original work of the team.

Policy: For the final submission (both for the demonstration and the technical report), the use of external, third-party libraries to implement functionality directly tied to the learning objectives is **not permitted**.

While teams are permitted to use such libraries during initial development for prototyping or comparative purposes, this code must be fully replaced with the team's own implementation in the final submitted work.

To provide clarity, the following examples are given:

Implementations that MUST be original (i.e., no libraries permitted):

- PID or other feedback control algorithms.
- Quadrature encoder reading and position tracking.
- Software-based switch debouncing.
- Finite State Machine (FSM) implementation.
- G-code command parsing.
- Trajectory generation (e.g., trapezoidal velocity profiles).

Use of foundational libraries that ARE permitted:

- The standard Arduino framework (Arduino.h, Serial, pinMode, analogWrite, etc.).
- The provided DFRobot motor shield library, if used solely for abstracting pin mappings and basic motor direction/speed commands. The core motion control logic must still be original.
- Basic mathematical libraries (e.g. cmath.h)

This list is illustrative, not exhaustive. It is the team's responsibility to ensure compliance with this policy. If there is any doubt as to whether the use of a particular library is permissible, it is mandatory to consult with the teaching staff for clarification before integrating it into your final design.

2.1.6 Resetting and Initialisation

The system should enter a stable IDLE state upon power-up, requiring no more than a single command (e.g., G28) to begin its operational sequence. It should return to a ready state after completing any motion command.

3 Design Considerations

The nature of the solution you design and implement is constrained by the hardware, but your approach to software, control, and analysis is open. Consider the following points.

3.1 Robustness and Reliability

Mechatronic systems are prone to errors from sensor noise, actuator limitations, software bugs, and mechanical inconsistencies. Design your software and control algorithms to be resilient to these real-world imperfections. Consider edge cases and potential failure modes.

3.2 Software Architecture

A well-structured program is easier to develop, debug, and maintain. The requirement to use an FSM is intended to guide you toward a robust architecture. Consider how to make your code modular, readable, and scalable.

3.3 System Integration and Debugging

You will be integrating hardware and software iteratively. Consider how your design facilitates testing. For example, implementing serial print statements to report sensor values, controller outputs, and system state can significantly speed up debugging.

3.4 Safety and Formal Analysis

The principles of engineering safety analysis are critical for any mechatronic system. A key part of this project requires you to look beyond the current hardware and analyse a future, more hazardous application. The HARA and FMEA are not just report-writing exercises; they are essential tools for identifying potential dangers and proposing mitigations before a system is built or modified. Your analysis will demonstrate your ability to think critically about system safety.

3.5 Version Control

While not formally assessed, it is **strongly recommended** that your team uses a version control system, such as Git (with a platform like GitHub or GitLab), to manage your source code. This is standard industry practice and will help you collaborate effectively, track changes, and recover from errors.

4 Resources

4.1 Lectures and Office Hours

The content of the lectures will roughly coincide with the point at which you should look to be at with the project at that time, and as such will include some topics that may be revision (hopefully) and some new topics. The material contained within the lectures will also inform the content of the exam. Any time left over at the end of the lecture may be utilised as office hours, in addition to the scheduled office hour on Monday 1-2 pm for the duration of the project.

4.2 Laboratory Sessions

Timetabled laboratory sessions are the primary time for system assembly, testing, and demonstration. Instructors and teaching assistants will be available to provide support. Access to oscilloscopes and benchtop power supplies is provided during these sessions.

4.3 Ed Discussion

This is the primary tool for asking questions that may benefit the entire class. Teaching staff will monitor the forum. For specific group issues, direct consultation during labs or office hours is preferred.

Please also note that Ed Discussion has been set so that any anonymous posts are not anonymous to the teaching staff, so please bear that in mind with regards to acceptable use etc.

4.4 Hardware Management and Faults

The provided hardware kits have been checked, but faults can still occur. It is the team's responsibility to test their components early and regularly. If you suspect a component is faulty, you must demonstrate the fault to a teaching staff member during a lab session. Upon verification, a replacement part will be issued. Teams are responsible for any damage to components caused by incorrect wiring or mishandling.

5 Assessment and Submission

5.1 Group System Demonstration (50 marks) - Due: Final Lab Session

During the final lab session, each group will demonstrate the functionality of their completed plotter. The demonstration will assess the successful and reliable execution of the functional requirements outlined in Section 2.1.4.

The demonstration will proceed as follows:

- Teams will be given the first 30 minutes of the session to set up. From that point on, the system should be ready for testing with minimal notice.
- The instructor will provide a short, previously unseen sequence of G1 and G28 commands to test the system's functionality and reliability.
- During the demonstration, the team will be asked questions about their design choices, control strategy, and software architecture.

5.2 Group Weekly Progress (5 marks) – Assessed: Weekly Lab Sessions

Team progress and process will be assessed via brief weekly stand-up meetings with an instructor during lab sessions. Further details on the procedure for these meetings are provided in the Project Lab & Process Guide.

5.3 Group Technical Report (40 marks)

Each group will submit a single, comprehensive technical report. The report should not exceed 3000 words (excluding appendices). It must include:

- 1) A title page and table of contents.
- 2) A high-level overview of the final hardware and software architecture (including FSM diagram).
- 3) A formal Safety Analysis for a hypothetical system upgrade. This analysis will form a significant part of your report's technical depth. The scenario to be analysed is as follows:
 - a) Scenario: The X-Y plotter is to be repurposed as a small-scale laser cutter by mounting a 2.5W Class 4 Diode Laser to the end effector.
 - b) Your analysis must include:
 - i) A Hazard and Risk Analysis (HARA) identifying the new hazards introduced by the laser and analysing the associated risks.
 - ii) A Failure Mode and Effects Analysis (FMEA) on at least two critical components or subsystems of the complete laser cutter system (e.g., the control software, a limit switch, the laser power supply).
 - iii) A discussion of proposed risk mitigations that would be necessary to make this system safer.
- 4) A Performance Evaluation section with telemetry data (e.g., graphs of step response) validating PID controller performance.
- 5) Conclusions and discussion of potential future improvements.
- 6) Appendices (at a minimum):
 - a) Well-commented source code.
 - b) Statement of contributions to the team project.

5.4 Individual Reflection (5 marks)

Each team member will submit a confidential individual reflection (300-500 words) addressing their primary contribution, a significant technical challenge faced, and key engineering principles learned.

5.5 Peer Evaluation

While it is expected that any issues with group dynamics and workload will be raised with the teaching staff and addressed early on during the project, there may be situations where this is not the case. A peer evaluation form will be used at the end

of the project to help ameliorate some of the effects of this so as to better weight marks according to contribution. The form that will be used, together with how it works is provided on Canvas.

6 Suggested Reading List

- Lecture notes in courses such as MECHENG 201, MECHENG 270, MECHENG 313, MECHENG 322 and MECHENG 370.
- Documentation as provided on Canvas